Exercício: Manipulando Conjuntos (Set) em Java

Objetivo:

O objetivo deste exercício é praticar o uso da interface `Set` para armazenar elementos únicos em Java.

Instruções:

1. Crie uma classe chamada `SetExercise`.
2. No método `main`, declare e inicialize um conjunto (set) de números inteiros.
3. Adicione alguns números inteiros ao conjunto.
4. Verifique se um número específico está presente no conjunto.
5. Remova um número do conjunto.
6. Imprima todos os números restantes no conjunto.

Passos para fazê-lo:

1. Crie a classe `SetExercise`.
2. Declare uma variável do tipo `Set<Integer>` e inicialize-a com um conjunto vazio.
3. Adicione alguns números inteiros usando o método `add`.
4. Use o método `contains` para verificar se um número específico está presente no conjunto.
5. Remova um número usando o método `remove`.
6. Utilize a representação padrão do conjunto na impressão.

Código do Exercício:

```
import java.util.HashSet;
import java.util.Set;

public class SetExercise {
    public static void main(String[] args) {
        // Inicialização do conjunto
        Set<Integer> numeros = new HashSet<>();

        // Adicionando números ao conjunto
        numeros.add(10);
        numeros.add(20);
        numeros.add(30);

        // Verificando se o número 20 está presente
        boolean contemNumero20 = numeros.contains(20);
        System.out.println("O conjunto contém o número 20? " +
contemNumero20);

        // Removendo o número 10
        numeros.remove(10);

        // Imprimindo os números restantes
        System.out.println("Números no conjunto: " + numeros);
    }
}
```

Explicação do Código:

- Criamos um conjunto (`Set`) chamado `numeros` para armazenar números inteiros.
- Adicionamos três números (10, 20 e 30) ao conjunto.
- Verificamos se o número 20 está presente no conjunto.
- Removemos o número 10 do conjunto.
- Imprimimos os números restantes no conjunto.

Exercício: Utilizando `Map` para Controle de Estoque

Objetivo

O objetivo deste exercício é entender e aplicar conceitos de **coleções** em Java, especificamente o uso de `Map` para gerenciar um estoque de produtos.

Instruções

1. Crie um programa que utilize um `Map` para representar o estoque de produtos.
2. Adicione produtos ao estoque com códigos e quantidades.
3. Implemente a verificação da quantidade disponível de um produto específico.
4. Implemente a remoção de uma quantidade específica de um produto do estoque.

Passos para Fazer o Exercício

1. **Importe as classes necessárias**:
   - Utilize `HashMap` e `Map` da biblioteca `java.util`.
2. **Crie o método `main`**:
   - Inicialize um `Map` para representar o estoque de produtos.
   - Adicione produtos ao estoque com códigos e quantidades.
   - Verifique a quantidade disponível de um produto específico.
   - Remova uma quantidade específica de um produto do estoque e atualize o estoque.
3. **Teste a funcionalidade**:
   - Execute o programa e verifique se as operações de adição, verificação e remoção estão funcionando corretamente.

Código do Exercício

```
import java.util.HashMap;
import java.util.Map;

public class ControleEstoque {
    public static void main(String[] args) {
        // Cria um Map para representar o estoque de produtos
        Map<Integer, Integer> estoque = new HashMap<>();

        // Adiciona produtos ao estoque
        estoque.put(123, 50);
        estoque.put(456, 30);
        estoque.put(789, 20);

        // Verifica a quantidade disponível de um produto
        int codigoProduto = 123;
        int quantidadeProduto = estoque.get(codigoProduto);
        System.out.println("Quantidade de produto " + codigoProduto +
": " + quantidadeProduto);

        // Remove alguns produtos do estoque
        int quantidadeRemovida = 10;
        if (quantidadeProduto >= quantidadeRemovida) {
            estoque.put(codigoProduto, quantidadeProduto -
quantidadeRemovida);
            System.out.println("Quantidade de produto " +
codigoProduto + " após remoção: " + (quantidadeProduto -
quantidadeRemovida));
        } else {
            System.out.println("Quantidade insuficiente para
remover.");
        }
    }
}
```

Explicação do Código

1. **Importação de Bibliotecas**:
    - `HashMap` e `Map` são importados da biblioteca `java.util` para criar e manipular o estoque de produtos.
2. **Inicialização do Estoque**:
    - Um `Map` é criado para representar o estoque, onde a chave é o código do produto (um `Integer`) e o valor é a quantidade disponível (também um `Integer`).
3. **Adição de Produtos**:
    - Produtos são adicionados ao estoque usando o método `put`, que associa um código de produto a uma quantidade.

4. **Verificação de Quantidade**:
    - A quantidade disponível de um produto específico é verificada usando o método `get`, que retorna o valor associado à chave fornecida.
5. **Remoção de Produtos**:
    - A quantidade de um produto é reduzida se a quantidade disponível for suficiente. O método `put` é usado novamente para atualizar a quantidade no estoque.
6. **Saída de Informações**:
    - Mensagens são impressas no console para mostrar a quantidade de produtos antes e depois da remoção.

Este exercício ajuda a entender como usar coleções em Java para gerenciar dados de forma eficiente.

Exercício: Utilizando List para Manipular Lista de Filmes

Objetivo

O objetivo deste exercício é entender e aplicar conceitos de **listas** em Java, utilizando a classe `ArrayList` para gerenciar uma lista de filmes.

Instruções

1. Crie um programa que utilize uma `ArrayList` para armazenar uma lista de filmes.
2. Adicione filmes à lista.
3. Implemente a verificação da presença de um filme específico na lista.
4. Imprima a lista de filmes e o resultado da verificação.

Passos para Fazer o Exercício

1. **Importe as classes necessárias**:
   - Utilize `ArrayList` e `List` da biblioteca `java.util`.
2. **Crie o método `main`**:
   - Inicialize uma `ArrayList` para armazenar os filmes.
   - Adicione filmes à lista.
   - Imprima a lista de filmes.
   - Verifique se um filme específico está na lista e imprima o resultado.
3. **Teste a funcionalidade**:
   - Execute o programa e verifique se as operações de adição, impressão e verificação estão funcionando corretamente.

Código do Exercício

```java
import java.util.ArrayList;
import java.util.List;

public class ListaFilmes {
    public static void main(String[] args) {
        List<String> filmes = new ArrayList<>();

        // Adiciona filmes à lista
        filmes.add("O Poderoso Chefão");
        filmes.add("Matrix");
        filmes.add("Interestelar");

        // Imprime a lista de filmes
        System.out.println("Lista de filmes:");
        for (String filme : filmes) {
            System.out.println(filme);
        }

        // Verifica se um filme específico está na lista
        String filmeBuscado = "Matrix";
        if (filmes.contains(filmeBuscado)) {
            System.out.println(filmeBuscado + " está na lista.");
        } else {
            System.out.println(filmeBuscado + " não está na lista.");
        }
    }
}
```

Explicação do Código

1. **Importação de Bibliotecas**:
    - `ArrayList` e `List` são importados da biblioteca `java.util` para criar e manipular a lista de filmes.
2. **Inicialização da Lista de Filmes**:
    - Um `ArrayList` é criado para armazenar os filmes, onde cada filme é representado por uma `String`.
3. **Adição de Filmes**:
    - Filmes são adicionados à lista usando o método `add`.
4. **Impressão da Lista de Filmes**:
    - A lista de filmes é impressa usando um loop `for-each` que itera sobre cada filme na lista.

5. **Verificação de Presença de um Filme**:
    - A presença de um filme específico na lista é verificada usando o método `contains`, que retorna `true` se o filme estiver na lista e `false` caso contrário.
6. **Saída de Informações**:
    - Mensagens são impressas no console para mostrar a lista de filmes e o resultado da verificação da presença do filme buscado.